LANGCHAIN MINI-PROJECT

PyMentor AI

AI Question-Answering Chatbot - Python Doubt Solver

Project Title	AI Question-Answering Chatbot
Selected Topic	Python Doubt Solver
Prepared By	Saksham Jadhav
Language Model	Google Gemini 3.8 Flash
Framework	LangChain
Language	Python

1. Objective

The objective is to build a simple chatbot using LangChain that accepts a question from the user and generates an AI answer. PyMentor AI applies this requirement to Python learning, where the user can ask conceptual, syntax, code, or debugging questions.

2. Project Overview

PyMentor AI is a console-based educational chatbot. The program validates a question, inserts it into a reusable LangChain prompt template, sends the formatted prompt to Google Gemini, parses the model response into plain text, and prints the answer. The interaction continues until the user types exit or quit.

3. Technologies Used

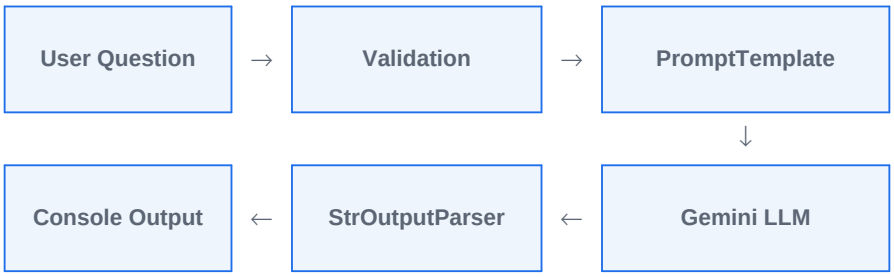
• **Python** - application logic, console input and error handling.

• **LangChain** - prompt templating and model pipeline orchestration.

• **Google Gemini** - language model that generates the answer.

• **python-dotenv** - loads the API key and model configuration from .env.

4. System Workflow



5. LangChain Implementation

The application creates a reusable prompt with **PromptTemplate.from_template()**. The Gemini model is connected with **ChatGoogleGenerativeAI**. LangChain Runnable composition then builds a compact pipeline:

```
chain = prompt | llm | StrOutputParser()
```

For every question, the program calls **chain.invoke({"question": cleaned_question})**. This passes the user input through the prompt, the language model, and the output parser in sequence.

6. Prompt Design

The prompt defines PyMentor AI as a patient Python study assistant. It instructs the model to answer clearly, include short examples when useful, explain common mistakes, avoid inventing code output, and remain within a Python/programming scope. The user input is inserted through the **{question}** variable.

7. Security and Error Handling

• The Gemini API key is not hard-coded; it is loaded from the **.env** file.

• The **.gitignore** file excludes .env so a real secret is not accidentally committed.

• Empty questions and questions longer than 2000 characters are rejected.

• Missing dependencies and missing API credentials produce friendly setup errors.

• Network, quota, API, model, and empty-response failures are handled without exposing a raw traceback to the user.

8. Required Five-Question Test Set

No.	Question	Expected coverage
1	What is a Python list and how is it different from a tuple?	Mutability, ordering, examples
2	Explain the difference between = and == in Python.	Assignment vs comparison
3	Write a Python function to check whether a number is prime.	Correct function and edge cases
4	What does range(1, 6) produce and why?	1 to 5; stop is excluded
5	Why do I get TypeError when joining a string and an integer?	Type mismatch and conversion fixes

9. Verification Performed

The project was statically compiled and an offline self-test was executed. The self-test verified required files, valid Python syntax, declared dependencies, the prompt variable, the .env key variable, five question inputs through the application function, empty-input rejection, and overlong-input rejection. All offline tests passed. A live Gemini call requires a valid private API key, so no credential was fabricated or exposed in the submission package.

Offline verification result	**PASS**
Live Gemini API call	Ready after adding a valid private API key

10. Conclusion

PyMentor AI satisfies the mini-project requirements with a clear LangChain pipeline, Google Gemini integration, reusable prompt template, user input, AI output, secure .env configuration, and basic error handling. The code is organized for a simple classroom demonstration while remaining easy to extend with a GUI, memory, RAG, or additional study features.

End of Report